

0.96.1. Une lettre qui tire — un tir, tout seul, puis plusieurs

Le 0.96, mais le A tire tout seul, sans fin : un tir toutes les 600 ms. Trois tirs volent en même temps : leurs colonnes sont rangées dans un tableau, `tx[3]`, et `vole[3]` dit lesquels sont en vol.

1. Ce qu'il faut comprendre

C'est le 0.96, avec plusieurs tirs. Le A tire **tout seul**, toutes les 600 ms, sans jamais s'arrêter. Un tir met 1,7 seconde à traverser l'écran : pendant ce temps, d'autres sont partis. On en voit **trois** en même temps sur la ligne.

Le problème : au 0.96, une seule variable `tx` gardait LA colonne du tir. Trois tirs, ce sont trois colonnes. On pourrait écrire `tx1`, `tx2`, `tx3`... et recopier trois fois le même code. Non : on les range dans un **tableau**, comme la queue du serpent (le 0.91.6).

Ce qui est nouveau ici (1) : deux tableaux. `uint8_t tx[3];` réserve **trois** colonnes : `tx[0]`, `tx[1]` et `tx[2]` (on compte depuis 0). `uint8_t vole[3];` dit, pour chacune, si le tir est en vol (1) ou libre (0). Au départ, tout vaut 0 : aucun tir ne vole.

Ce qui est nouveau ici (2) : lancer dans une case libre. Toutes les 600 ms (`chaque(600)`), une boucle `for` passe sur les trois tirs et lance le **premier** qui ne vole pas : `vole[i] = 1`, `tx[i] = 3`. La variable `lance` passe à 1 dès qu'un tir est parti : les suivants, même libres, ne partent pas à ce tour-ci. Un seul tir à la fois sort du A.

Ce qui est nouveau ici (3) : faire avancer chaque tir. Toutes les 100 ms, une autre boucle passe sur les trois : si `vole[i]` vaut 1, c'est **le pas du 0.96**, avec `tx[i]` au lieu de `tx`. Quand un tir sort de l'écran, `vole[i] = 0` : sa case du tableau est **libre**, elle servira au prochain tir.

Déroulons : au départ, `vole` = {0, 0, 0}. 600 ms : le tir 0 part (`tx[0] = 3`). 1,2 s : le tir 0 est déjà en colonne 9 ; le 1 part. 1,8 s : le 0 est sorti, `vole[0]` revient à 0 ; le 2 part. 2,4 s : la case 0 est libre, c'est elle qui repart. Les trois cases tournent sans fin.

Essaie : `chaque(300)` : plus de tirs voudraient partir, mais les trois cases sont souvent pleines, et il y a des trous dans la rafale. Passe alors à `tx[5]` et `vole[5]`, et `i < 5` dans les deux boucles.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <chaque>` → [0.29.3. La fonction chaque\(\) — toutes les n millisecondes](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <effacer>` → [0.16.3. La fonction effacer\(\) — effacer des cases](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **612 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l’a demandé — et ce qu’il a laissé, parce que rien ne l’appelle :

Gravé dans la cartouche :

démarrage	52 o	← la console, à l’allumage – la mise en route de la console
chaque_minuteur()	124 o	← main() – une fonction fournie par le langage
main()	252 o	← démarrage – ta fonction
VBlank	8 o	← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage	28 o	← image() – attend l’image suivante
AttendreVBlank	16 o	← démarrage – attend que l’écran soit libre
AttendreAcces	12 o	← EffacerCases, EcrireTexte – attend de pouvoir écrire une case
AdresseCase	10 o	← texte(), effacer() – trouve l’adresse d’une case de l’écran
EffacerCases	12 o	← effacer() – efface des cases
EcrireTexte	10 o	← texte() – écrit un texte, lettre par lettre
CopierTuiles	40 o	← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage – vide les deux cartes de l’écran
Tuiles	32 o	← CopierTuiles – 2 lettres sur 43
","	1 o	← texte() – un texte gravé
","	1 o	← texte() – un texte gravé

Lettres gravées : A.

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

texte()	× 2	48 o sur place, dans main() + AdresseCase (10 o), EcrireTexte (10 o)
effacer()	× 1	21 o sur place, dans main() + AdresseCase (10 o), EffacerCases (12 o)
poser()	× 1	6 o sur place, dans main()
image()	× 1	3 o sur place, dans main() + AttendreImage (28 o)
images()	× 1	2 o sur place, dans chaque_minuteur()

Tes fonctions appelées :

chaque_minuteur()	× 2	21 o sur place, dans main()
-------------------	-----	-----------------------------

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- UNE LETTRE QUI TIRE : un tir, tout seul – plusieurs tirs ----
// Le 0.96, mais le A tire TOUT SEUL, sans fin : un nouveau tir toutes les
// 600 ms. Plusieurs tirs volent en même temps : leurs colonnes sont rangées
// dans un TABLEAU (comme la queue du serpent, le 0.91.6).

// ---- NOUVEAU : trois tirs, rangés dans deux tableaux ----
// uint8_t tx[3] : TROIS cases, tx[0], tx[1] et tx[2] (on compte depuis 0).
// tx[i] : la colonne du tir numéro i ; vole[i] : 1 s'il est en vol.
// Au départ, toutes les cases valent 0 : aucun tir ne vole.

#include <poser>      // pose une tuile sur une case du fond
#include <ALPHABET>   // les lettres de la police : ALPHABET[0] est le A
#include <chaque>     // répond 1 toutes les n millisecondes
#include <texte>      // écrit un texte à l'écran
#include <effacer>    // efface des cases, ou tout le fond
```

2. UN TABLEAU — TX

```
uint8_t tx[3];
```

3. UN TABLEAU — VOLE

```
uint8_t vole[3];
```

4. UNE VARIABLE — LANCE

```
uint8_t lance = 0;          // 1 = un tir vient de partir, à ce tour-ci
```

5. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    poser(2, 8, ALPHABET[0]); // la lettre A, celle qui tire : (2, 8)

    while (true) {
        image();

        // ---- NOUVEAU : un tir part toutes les 600 ms, dans une case LIBRE ----
        // La boucle regarde les trois tirs ; le premier qui ne vole pas part.
        // lance passe à 1 : les suivants, même libres, attendent le prochain tour.
        if (chaque(600)) {
            lance = 0;
            for (uint8_t i = 0; i < 3; i++) { // i vaut 0, puis 1, puis 2
                if (lance == 0 && vole[i] == 0) {
                    vole[i] = 1; // le tir n° i est en vol...
                    tx[i] = 3; // ...il part juste à droite du A...
                    texte(tx[i], 8, "."); // ...et on le dessine
                    lance = 1;
                }
            }
        }

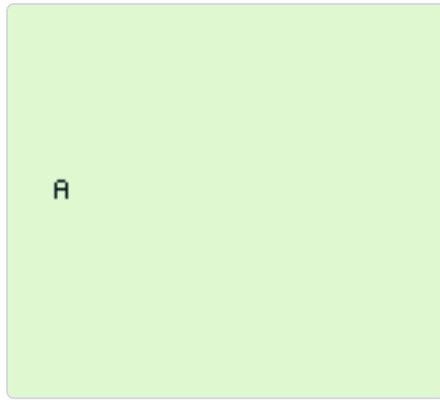
        // ---- CHAQUE tir avance : le pas du 0.96, dans une boucle ----
        // Le même code, avec tx[i] au lieu de tx, et vole[i] pour savoir
        // s'il faut s'en occuper.
        if (chaque(100)) {
            for (uint8_t i = 0; i < 3; i++) {
                if (vole[i] == 1) {
                    effacer(tx[i], 8, 1); // 1. il quitte sa case
                    tx[i] = tx[i] + 1; // 2. une colonne plus loin
                    if (tx[i] < 20) { // 3. encore dans l'écran : on le dessine
                        texte(tx[i], 8, ".");
                    } else { // sorti : sa case du tableau est libre,
                        vole[i] = 0; // elle servira au prochain tir
                    }
                }
            }
        }
    }
}
```

3. Ce qui se passe, étape par étape

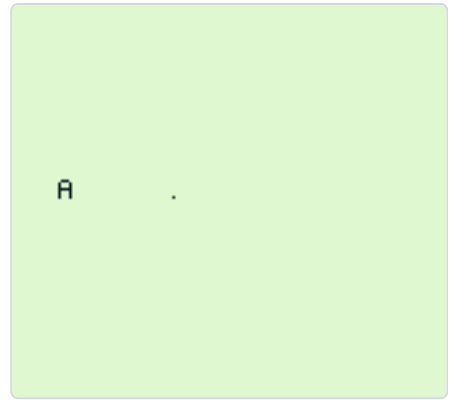
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



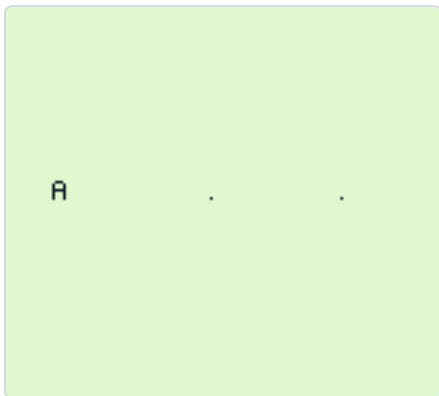
la toute première image — la console vient de démarrer — image 1



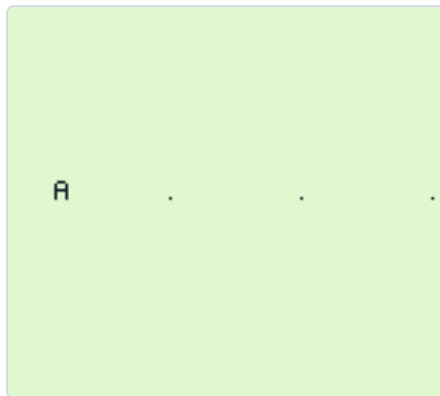
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

Le A en (2, 8) tire tout seul, un « . » toutes les 600 ms ; trois tirs volent en même temps sur la ligne 8, et chacun disparaît au bord.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le A reste en (2, 8)
- ✓ plusieurs tirs volent en même temps sur la ligne 8, jamais plus de trois (au plus 3)

Ensuite : **0.97. Une lettre qui tire — A lance le tir** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++